Estructura del proyecto

El repositorio es un proyecto Maven multimódulo (Java 21) con cuatro módulos: `common`, `broker`, `sistema-centralizado` (Sistema 1) y `sistema-cliente` (Sistema 2). Cada módulo ejecutable tiene su propio `Dockerfile`, y `docker-compose.yml` los orquesta junto con MariaDB.

Vista general

```
/
├── .githooks/                # Hook pre-commit: compila la documentación
├── .run/                     # Configuraciones de ejecución del IDE
├── bin/                      # Mermaid CLI local para generar documentos
├── broker/                   # Módulo Broker (intermediario)
├── common/                   # Módulo compartido: contrato JSON y modelos
├── docker/
│   └── mariadb/init/         # Scripts SQL de inicialización
│       ├── 01_schema.sql     # Esquema de la base de datos
│       └── 02_seed.sql       # Datos iniciales
├── docs/                     # Documentación del proyecto
├── scripts/docs/             # Generador de PDFs a partir de Markdown y
Mermaid
├── sistema-centralizado/     # Módulo Sistema 1 (proveedor de servicios)
├── sistema-cliente/          # Módulo Sistema 2 (cliente Swing)
├── .env.example              # Plantilla de variables de entorno
├── docker-compose.yml        # Orquestación de MariaDB, Broker y sistemas
├── pom.xml                   # POM padre del proyecto multimódulo
└── README.md
```

Todos los módulos siguen el layout estándar de Maven (`src/main/java`, `src/main/resources`, `src/test/java`) bajo el paquete base `mx.uady.inventario`.

Módulo `common`

Biblioteca que comparten los tres componentes. Define el contrato de mensajes para que el cliente, el Broker y el proveedor hablen el mismo JSON.

```
common/src/main/java/mx/uady/inventario/common/
├── contrato/     # Solicitud, Respuesta y EstadoRespuesta
```

```
├── error/        # CodigoError: códigos de error del contrato
├── json/         # GsonFactory y adaptadores (LocalDateTime)
├── modelo/       # Producto, Mercancia, Venta, Usuario, TiendaAlmacen, ...
└── servicio/     # Servicios y Operaciones: nombres de servicios y
operaciones
```

Los servicios definidos son `gestorTiendas`, `inventario` y `gestorPrecios`.

Módulo `broker`

Intermediario que registra a los proveedores y enruta cada solicitud al proveedor que ofrece el servicio pedido.

```
broker/src/main/java/mx/uady/inventario/broker/
├── Main.java     # Punto de entrada
├── model/        # Proveedor: datos de un proveedor registrado
├── network/      # BrokerServer y ManejadorConexion (un hilo por conexión)
├── registry/     # RegistroServicios: servicio -> proveedores
├── routing/      # Enrutador: reenvío, timeouts y reintentos
└── util/         # Configuración por variables de entorno, bitácora y JSON
```

Módulo `sistema-centralizado` (Sistema 1)

Proveedor de servicios sobre MariaDB. Atiende las solicitudes que llegan del Broker y, cuando hay pantalla, abre la interfaz Swing del Administrador. Sigue una arquitectura por capas (Modelo-Servicio-Repositorio) con MVC para la interfaz.

```
sistema-centralizado/src/main/java/mx/uady/inventario/central/
├── Main.java         # Arranca el servidor TCP y, si hay pantalla, la
interfaz
├── controller/       # Controladores de la interfaz y tareas en segundo
plano
├── model/            # Modelos propios de la vista
(ProductoConUnidades, ...)
├── network/          # Servidor TCP, despacho de operaciones y registro en
el Broker
├── repository/       # Acceso a datos por JDBC
├── service/          # Lógica de negocio, validaciones y generación de QR
│   └── jdbc/         # Implementaciones JDBC de los servicios y
transacciones
├── util/             # Configuración de BD y del servicio, conexión y QR
```

```
└── view/             # Ventana principal y paneles de Productos, Mercancía
y Ventas
    └── components/   # Componentes reutilizables (vista previa del QR)
```

Módulo `sistema-cliente` (Sistema 2)

Aplicación de escritorio para el proveedor. Se comunica con el Sistema 1 únicamente a través del Broker.

```
sistema-cliente/src/main/java/mx/uady/inventario/cliente/
├── Main.java         # Punto de entrada
├── controller/       # RegistroVentaController: flujo de registro de
ventas
├── model/            # ClienteTCP y DTOs (Mercancia, Tienda, Usuario)
├── network/          # Reservado para la capa de red
├── util/             # Configuración de red y traducción de errores
└── view/             # RegistroVentaFrame y filtros de mercancía
    └── components/   # Componentes reutilizables (vista previa del QR)
```

Documentación (`docs/` y `scripts/docs/`)

```
docs/
├── 01_documentacion_inicial/   # Diseño del sistema (entrega inicial)
└── 02_documentacion_final/     # Documentación final y evidencias
    ├── diagramas/              # Diagramas Mermaid (.mmd)
    ├── evidencias/             # Salidas y scripts de las pruebas
    ├── extras/                 # Material de referencia externo
    ├── markdown/               # Un subdirectorio por documento
    └── out/                    # PDFs y diagramas generados
```

Los PDFs de `out/` se generan con los scripts de `scripts/docs/`, ya sea con Docker (`procesar-docs-docker`) o con un entorno local (`setup-local` y `procesar-docs-local`). El hook `.githooks/pre-commit` los regenera antes de cada commit.

Despliegue

Guia para levantar MariaDB, el Broker y el Sistema 1 con Docker Compose, y para abrir el cliente Swing (Sistema 2) desde Windows.

> Estado actual: el `Main` del cliente es un esqueleto que solo imprime un mensaje de arranque. Las secciones sobre el cliente describen como ejecutarlo hoy y como abrirlo cuando la interfaz Swing exista; no hay pantalla que mostrar todavia.

Requisitos

- Docker Desktop con Docker Compose v2 (`docker compose version`).
- JDK 21 o superior y Maven 3.9+, solo si vas a compilar fuera de Docker. El `pom.xml` compila con `release 21`.
- Para el cliente en Windows: un servidor X11 (WSLg, VcXsrv o X410) o ejecutarlo directamente desde el IDE.

1. Configurar las credenciales

Las credenciales viven en `.env`, que esta ignorado por git. Parte de la plantilla:

```
Copy-Item .env.example .env
```

```
cp .env.example .env
```

Edita `.env` y reemplaza los marcadores:

- `DB_ROOT_PASSWORD`: contrasena del usuario `root` de MariaDB.
- `DB_PASSWORD`: contrasena del usuario de la aplicacion (`DB_USER`).

Ambas son **obligatorias**. Si falta alguna, `docker compose` se detiene con un mensaje como:

```
error while interpolating services.mariadb.environment.MARIADB_PASSWORD:
required variable DB_PASSWORD is missing a value: define DB_PASSWORD
en .env
```

No uses las contrasenas de ejemplo en un entorno real y no subas `.env` al repo.

2. Levantar el stack

```
docker compose up --build
```

El arranque respeta este orden:

1. `mariadb` arranca y espera a que su healthcheck (`mariadb-admin ping`) pase.
2. `broker` arranca y espera a que su healthcheck (`nc -z 127.0.0.1 $BROKER_PORT`) confirme que escucha.
3. `sistema-centralizado` arranca cuando MariaDB y el broker estan `healthy` , y se registra en el broker con `SERVICE_HOST=sistema-centralizado` .

Para dejarlo en segundo plano:

```
docker compose up --build -d
docker compose ps
```

`docker compose ps` debe mostrar `uady-mariadb` , `uady-broker` y `uady-sistema-centralizado` como `running` / `healthy` .

3. Verificar el orden y el registro

```
docker compose logs broker
docker compose logs sistema-centralizado
```

- El broker imprime `Broker iniciado en el puerto 5000 ...` con sus timeouts.
- El Sistema 1 imprime `Escuchando en el puerto 6000 ...` y luego `Proveedor registrado en el broker broker:5000 como sistema-centralizado (gestorTiendas, inventario, gestorPrecios)` .

Si el broker tarda en estar listo, el Sistema 1 reintenta con `REGISTRO_INTENTOS` y `REGISTRO_ESPERA_MS` sin dejar de escuchar.

4. Puertos y variables

Componente	Puerto host	Variables principales
MariaDB	3306	`DB_ROOT_PASSWORD`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`
Broker	`BROKER_PORT` (5000)	`BROKER_TIMEOUT_*`, `BROKER_REINTENTOS`, `BROKER_FALLOS_PARA_INDISPONIBLE`, `BROKER_INDISPONIBLE_MS`
Sistema 1	`SERVICE_PORT` (6000)	`DB_*`, `BROKER_HOST`, `BROKER_PORT`, `SERVICE_HOST`, `SERVICE_POOL_SIZE`, `REGISTRO_*`, `QR_DIR`

`BROKER_PORT` y `SERVICE_PORT` se usan tanto para el puerto interno como para el mapeo al host; si los cambias en `.env`, ambos lados quedan alineados.

5. Cliente Swing en Windows

Windows no trae servidor X11, y una aplicacion Swing necesita uno. Hay dos caminos.

Opcion A: desde el IDE (recomendada para desarrollo)

1. Abre el proyecto en IntelliJ IDEA, Eclipse o VS Code.
2. Configura el proyecto con **JDK 21**. Las configuraciones de `.run/*.xml` del broker y del cliente apuntan al JRE 17; si las usas, cambia el JRE a 21 (el `pom.xml` compila con 21).
3. Crea una configuracion de ejecucion para `mx.uady.inventario.cliente.Main` con estas variables de entorno, apuntando al broker publicado por Docker:
4. `BROKER_HOST=localhost`

5. `BROKER_PORT=5000`
6. Ejecuta. La ventana Swing se dibuja directamente en el escritorio de Windows, sin X11 ni contenedor.

Tambien puedes compilar el fat-jar y ejecutarlo en Windows:

```
mvn -pl sistema-cliente -am clean package
java -jar sistema-cliente/target/sistema-cliente.jar
```

Opcion B: en Docker con un servidor X11

Util cuando quieres el cliente dentro de la misma red de Docker. El compose ya trae el perfil `client` con la imagen que incluye las librerias X11.

Con WSLg (Windows 11 + WSL2): ejecuta desde una terminal WSL2 con integracion de Docker Desktop. El socket `/tmp/.X11-unix` se comparte automaticamente:

```
export DISPLAY=:0
docker compose --profile client up --build
```

Con VcXsrv o X410 (Windows nativo): inicia el servidor X permitiendo conexiones TCP (en XLaunch: Multiple windows, Start no client, marca Disable access control). Luego, en PowerShell:

```
$env:DISPLAY = "host.docker.internal:0.0"
docker compose --profile client up --build
```

El montaje `/tmp/.X11-unix` de `docker-compose.yml` solo se usa en WSL/Linux; en Windows nativo el cliente se conecta por TCP usando `DISPLAY`. Si el Firewall de Windows pregunta, permite la conexion del servidor X.

6. Interfaz del Administrador (Sistema 1)

El Sistema 1 abre su interfaz Swing (Productos, Mercancía y Ventas) solo cuando hay pantalla; dentro de Docker arranca únicamente como servicio. Para usarla, ejecútalo desde el IDE en Windows, con MariaDB levantada en Docker.

1. Levanta la base de datos: `docker compose up -d mariadb`. Si el puerto 3306 está ocupado por otra base local, publícala en otro puerto y ajusta `DB_PORT` en el paso 2.
2. Crea una configuración de ejecución para `mx.uady.inventario.central.Main` con **JDK 21** (`.run/sistema-centralizado.run.xml` ya usa el JRE 21) y estas variables:

Variable	Valor
`DB_HOST`	`localhost`
`DB_PORT`	`3306` (o el puerto publicado)
`DB_NAME` , `DB_USER` , `DB_PASSWORD`	los mismos de `.env`
`QR_DIR`	`storage/qr` (carpeta local; ignorada por git)
`BROKER_HOST` / `SERVICE_HOST`	`localhost`

En VS Code puede ser un `.vscode/launch.json` (la carpeta está ignorada por git):

`jsonc { "type": "java", "name": "Sistema 1 (interfaz)", "request": "launch", "mainClass": "mx.uady.inventario.central.Main", "projectName": "sistema-centralizado", "envFile": "${workspaceFolder}/.env", "env": { "DB_HOST": "localhost", "BROKER_HOST": "localhost", "SERVICE_HOST": "localhost", "QR_DIR": "storage/qr" } }` 3. Ejecuta. El proceso también levanta el servidor TCP en `SERVICE_PORT` ; si el contenedor `uady-sistema-centralizado` está corriendo, detenlo antes ( `docker compose stop sistema-centralizado`) para liberar el puerto 6000.

Notas:

- La interfaz y el servidor de red comparten los mismos servicios sobre MariaDB: una venta hecha por un cliente aparece en la pestaña Ventas al pulsar **Actualizar**.
- `QR_DIR` solo aplica al IDE. En Docker el Sistema 1 usa siempre `/app/storage/qr` (volumen `qr_storage`).
- Si la base de datos no responde, cada pantalla muestra "No se pudo conectar con la base de datos" y la ventana sigue respondiendo; pulsa **Actualizar** cuando vuelva a estar disponible.

7. Detener y limpiar

```
docker compose down          # detiene los contenedores y conserva los
datos
docker compose down -v       # ADVERTENCIA: borra el volumen de MariaDB
```

`docker compose down -v` es destructivo: al recrear el volumen se vuelven a ejecutar `01_schema.sql` y `02_seed.sql`, y se pierde cualquier dato agregado despues.